

Schedule Master- Release Summary

Instruction: If any of the following sections are well documented, please attach the URL and have a brief description of it. No need to repeat.

Team members

Name and email	GitHub id	Role of each member and tasks done by each member (brief description)
Ngoc Kien Mai maik@myumanitoba.ca	maingockien01	Full-stack and DevOps Develop and maintain infrastructure, CI/CD pipeline. Develop Roadmap features.
Jack Nguyen trungn@myumanitoba.ca	ntt2402vn	Full-stack developer Develop Course management features. Initialize database schema and seeding. Implement load testing and security scanner
Quang Anh Le leqa@myumanitoba.ca	AnhLe-Axel	Full-stack developer Develop Authentication features. Design and enhance FE.
Arshdeep Singh sahotaa1@myumanitoba.ca	arsh331	Full-stack developer Develop Calendar features.

Project summary

<https://team8-comp4350.azurewebsites.net/lookup>

Have you ever felt tired by the tedious process of managing your courses and schedule on school websites? With Aurora, it feels like we're constantly clicking through endless pages just to view our courses, add or drop classes, and check our schedules for the week. It's time-consuming, inefficient, and frankly, frustrating. That's why we have come up with Schedule Master, the solution to the chaos of traditional school websites like Aurora. With Schedule Master, you can effortlessly view your courses, add or drop classes, and check your weekly schedule—all from one central hub. No more back-and-forth navigation. It's streamlined, it's intuitive, and it's designed with the student in mind. Schedule Master goes beyond just managing your current courses. With customizable degree roadmaps, you can plan your academic journey with ease. Whether you're following a prescribed path or creating your own, Schedule Master empowers you to take control of your education. So let's bid farewell to the

frustrations of outdated school websites and embrace the future of academic management with Schedule Master.

GitHub repository link

<https://github.com/maingockien01/comp4350-team8>

DockerHub repository link

Link: <https://hub.docker.com/repository/docker/imkevinmai/team8/general>

Docker image: imkevinmai/team8:latest

Instructions to run docker images:

```
docker run -p 3001:3000 \  
-e PORT=3000 \  
-e DB_HOST={db_host} \  
-e DB_USER={db_user} \  
-e DB_PASSWORD={db_pwd} \  
-e DB_DATABASE={db_database} \  
imkevinmai/team8:latest \
```

We do not provide the details of database connection here for the sake of security.

List of user stories for each sprint

Sprint#	US#	User stories	Status
2	1	As a new user, I want to create an account so that I can manage my courses.	Done
	2	As a user, I want to sign into my account so that I can manage my courses.	Done
	3	As an user, I want to view my calendar so that I know my class time.	Done
	4	As a user, I want to view a roadmap of my degree so that I know which classes I should take.	Done

	5	As an user, I want to see which courses are available during a specific term so I can know which course can I add	Done
3	6	As an user, I want to export my timetable.	Done
	7	As an user, I want to see a course full information when I clicked on a specific course	Done
	8	As an user, I want to add/drop courses so that I can view them in my calendar	Done
	9	As a logged-in user, I want to change my account information so that I can have a profile that is up to date	Done
	10	As a user, I want to create my own customized roadmap.	Done
4	11	As a user, I want to always receive responses from the web application within 300 milliseconds so that I do not need to wait for too long.	Tested

User manual

Core Features:

ACCOUNT MANAGEMENT:

1. Signup:

Step 1: Open the website at [Schedule Master](#).

Step 2: Click on the "Register" button.

Step 3: Fill in valid information including username, full name, and password.

Step 4: Click on the "Signup" button.

2. Login:

Step 1: On the login page, enter the username and password provided during signup.

Step 2: Click on the "Login" button.

3. Change user profile:

Step 1: After logging in, click on the user icon.

Step 2: From the drop-down menu, select the "Profile" option.

Step 3: On the User Profile page, enter new information into one or more of the fields (e.g., new username and/or password and/or full name).

Step 4: If changing password, make sure that the password field and the confirm password field are the same.

Step 5: Click on the "Update" button.

ROADMAP AND PERSONAL ROADMAP:

1. Viewing Roadmap:

Step 1: Click on “Roadmap” button on the navigation bar

Step 2: In the drop down menu, choose the desired degree

Step 3: After chosen with a specific degree, a roadmap of recommended courses will appear, hover through each courses to see its name and prerequisite (pink highlighted courses are prerequisites for the blue highlighted course)

2. **Customize Personal Roadmap:**

Add courses to roadmap:

Step 1: Click on “Personal Roadmap” button on the navigation bar, this will navigate user to the page contains 2 areas: Personal Roadmap and Available Courses

Step 2: In the Available Courses drop box, choose the desired course, then click in it

Step 3: After clicked, the system will display detail information about that specific course, click “ADD TO ROADMAP” button, if eligible (user has all prerequisites of that specific course in their personal roadmap already), the course will appear on the left under the Personal Roadmap.

Step 4: Under the Personal Roadmap, click “SAVE” button to save it, else it will disappear.

Remove courses from roadmap:

Step 1: If users wish to delete a course from their personal roadmap, click the “X” circle next to the course that one wants to remove, if successfully remove (the course removed is not a prerequisite of another course in one’s personal roadmap), it will disappear from the personal roadmap.

Step 2: Click “SAVE” button to save it, else it will reappear.

COURSE MANAGEMENT:

4. **Viewing Courses:**

Step 1: Click on “Look Up Courses” button on the navigation bar

Step 2: Choose desired Degree and Term

Step 3: Click “Apply” button

Step 4: User should see a list of courses, scroll up and down to see all of them

Step 5: Click on a specific course (click in its rectangle area) to get to the Detail page of the course (a more specific and informative page about the course)

5. **Add Courses:**

1st approach: Through Look Up Courses

Step 1: Click on “Look Up Courses” button on the navigation bar, users will be navigated to a page which is waiting for them to select specific degree and term.

Step 2: Choose desired Degree and Term must be the current term (Fall 2024), if not current term, then users cannot add these courses to their calendar.

Step 3: Click “Apply” button

Step 4: User should see a list of courses, scroll up and down to see all of them

Step 5: Click on a specific course (click in its rectangle area) to get to the Detail page of the course (a more specific and informative page about the course)

Step 6: Check the Section area in the Detail page to find available sections, each section include

information such as Section ID, Location, Professor, Time.

Step 7: After choosing the desired section that user wants to register in, click the “Add Section” button next to that specific section, if added successfully, this will navigate user to the page which contains all courses registered by that user. Else, there will be an error notified to users.

2nd approach: Through Add/Drop Courses Page

Step 1: Click on “Add/Drop Courses” button on the navigation bar, users will be navigated to the page which contains all courses registered and an input box for adding courses.

Step 2: In the input box next to the “Add Section”, fill in the desired section id (to find a section id of a specific course, read **Step 6 in 1st Approach** above).

Step 3: Click “Add” button, if added successfully, the new course will appear under Registered Sections, else, there will be an error notified to users.

6. Drop Courses:

Step 1: Click on “Add/Drop Courses” button on the navigation bar, users will be navigated to the page which contains all courses registered and an input box for adding courses

Step 2: Under the Registered Sections, the list of courses are presented there, this represents active registration of the user. Next to each section in the Registered Sections area, there is a wastebasket icon.

Step 3: Delete a specific section by clicking the wastebasket icon next to it, a pop-up message will ask for confirmation, click “Drop” to drop the course, “Cancel” to cancel to action.

CALENDAR:

1. Add Courses:

Add the desired courses into your account using the steps mentioned in the Add courses section of the Course Management.

2. Navigate to calendar:

Step 1: Click on “Calendar” button on the navigation bar, user will be navigated to the calendar page.

Step 2: Users will be presented with the timings and locations of the courses on the page for all the active courses

Step 3: The page will automatically load the changes made to the current courses made by the user.

3. Export calendar:

Step 1: Verify the calendar page is selected in the Navbar.

Step 2: An “Export” button in the top right corner of the calendar page will be present, in case the calendar has courses it will be enabled or disabled in case of no active courses.

Step 3: Click on the “Export” button.

4. Export calendar:

Step 1: Verify the calendar page is selected in the Navbar.

Step 2: An “Export” button in the top right corner of the calendar page will be present, in case the calendar has courses it will be enabled or disabled in case of no active courses

Step 3: Click on the “Export” button

Step 4: User should see a new file with the name “schedule.ics” being downloaded

Step 5: User can add the .ics file to any calendar of their choice and a weekly schedule will be available for the appropriate term dates.

Overall Arch and Design

[https://github.com/maingockien01/comp4350-team8/wiki/Block-Diagram-\(Architecture\)](https://github.com/maingockien01/comp4350-team8/wiki/Block-Diagram-(Architecture))

Infrastructure

For each library, framework, database, tool, etc

Backend	NestJS, TypeORM
Frontend	React, MaterialUI
Database	MySQL
Infrastructure	Docker, docker-compose, Azure

Name and link

We opted for NestJS due to its robust support for rapid development and modularization. One of the primary reasons for selecting NestJS is its comprehensive suite of built-in tools catering to common functionalities like authentication, database management, and configurations. Leveraging these features allows us to allocate most of our time and resources towards enhancing our core business functionalities. Furthermore, our development approach aligns seamlessly with NestJS's modular philosophy, empowering individual developers to take ownership of specific features. This modular structure fosters collaboration while ensuring adherence to our established coding standards.

In the realm of frontend development, we chose React for its widespread adoption and extensive availability of online resources. By leveraging React, we anticipated quicker development cycles supported by a wealth of documentation readily accessible on the internet. Additionally, proficiency in React adds significant value to our developers' skill sets and enhances their marketability in the future job market.

For our backend database, MySQL emerged as the optimal choice due to the relational nature of our data. Renowned for its stability and robust community support, MySQL provides a reliable foundation for managing our data efficiently.

Streamlining our development workflow, we opted for GitHub Actions, consolidating our build, test, and deployment processes into a single, cohesive platform. This decision ensures simplicity and efficiency by centralizing our development tasks within one comprehensive toolset.

Furthermore, our choice of Azure as our cloud platform is driven by its favorable student subscription plan, offering cost-effective access to a wide array of cloud services. By utilizing Azure, we can leverage a scalable and reliable infrastructure to support our application's growth and development needs effectively.

Naming Conventions

List your naming conventions or just provide a link to the standard ones used online.

[Typescript style guide](#)

[Naming cheatsheet](#)

It's worth noting that the conventions in NestJS are aligned with many of the conventions commonly used in object-oriented programming (such as PascalCase for class names, camelCase for variable and function names), as well as some conventions from popular JavaScript frameworks like Angular and Express.js.

Code

Key files: top 5 most important files (full path). We will also be randomly checking the code quality of files. Please let us know if there are parts of the system that are stubs or are a prototype, so we grade these accordingly.

File path with a clickable GitHub link	Purpose (1 line description)
apps/backend/src/routes.module.ts	Define api routes in the backend
apps/backend/src/auth/auth.controller.ts	As an example of how controller is defined
apps/backend/src/entities/user.entity.ts	As an example of how entity is defined
apps/frontend/src/App.tsx	Define FE apps
apps/backend/src/app.module.ts	Define BE apps

Continuous Integration and deployment (CI/CD)

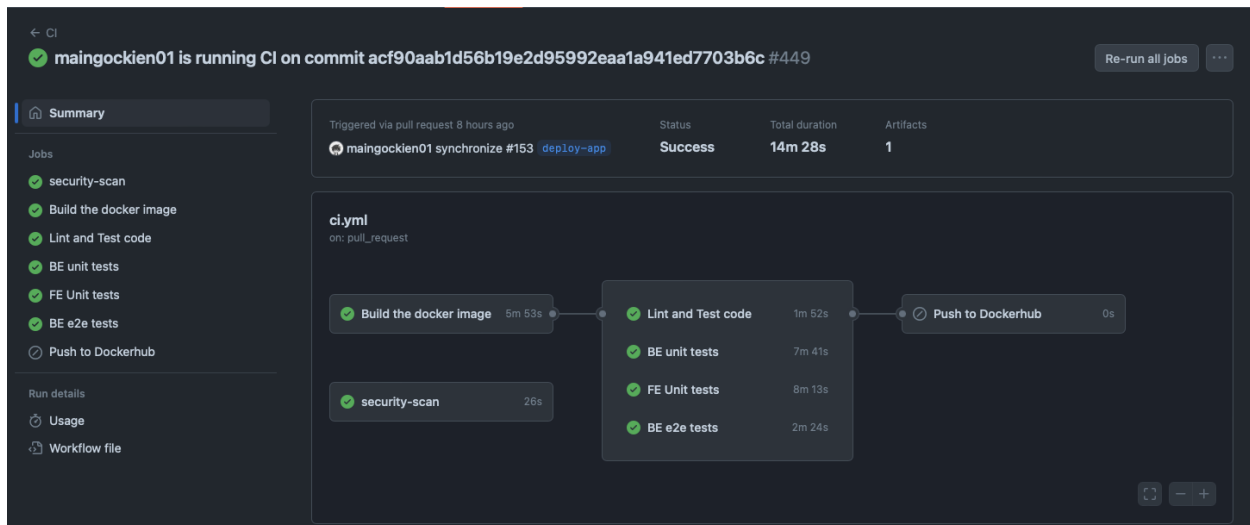
1) Describe your CI/CD environment and the clickable link to your CI/CD pipeline. For instance, if you use GitHub Action, provide the link to the workflow; if you use Jenkins, provide the link to the pipeline file.

Environment: GitHub Actions

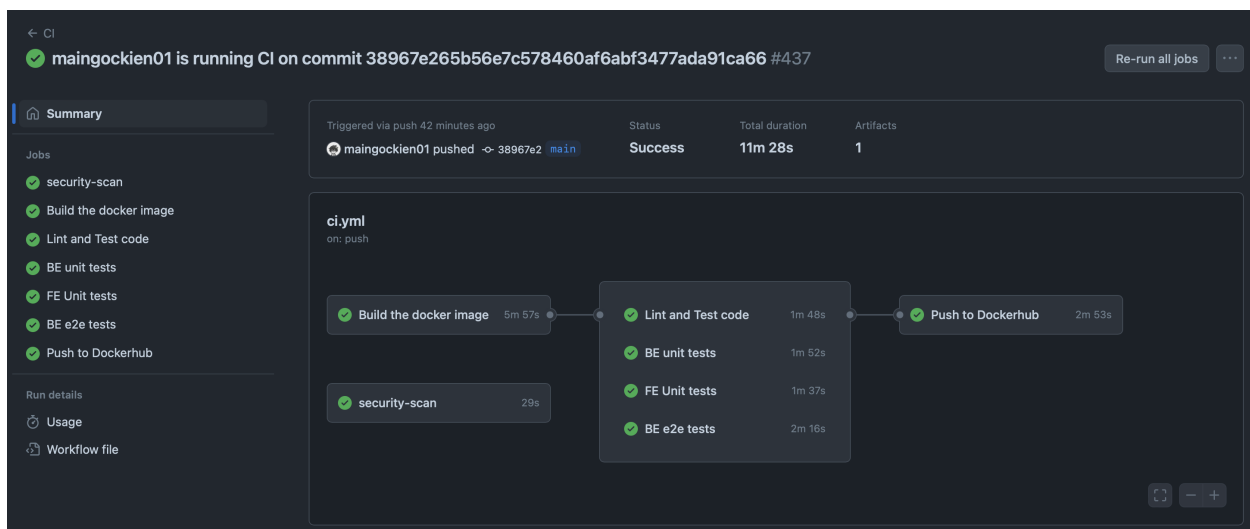
CI/CD: <https://github.com/maingockien01/comp4350-team8/blob/main/.github/workflows/ci.yml>

The pipeline push image to Docker Hub when the PR is merged to main.

2) Snapshots of the CI/CD execution. Provide one for CI and one for CD to demo you have successfully set up the environment.



CI: pipeline does not push on PR checks



CD: pipeline pushes image on main branch

Testing

Link to testing plan

Unit/integration/acceptance test

Each story needs a test before it is complete. In other words, the code coverage (in terms of statements) should be 100%. If some class/methods are missing unit tests, please describe why and how you are checking their quality. Please describe any unusual/unique aspects of your testing approach.

List the 10 most important unit tests with links below (if there are more than one unit tests in one test file, indicate clearly).

Test File path with clickable GitHub link	What is it testing (1 line description)
apps/backend/test/unit/auth/auth.controller.	Test the signUp() function in auth.controller.ts

unit.ts - at line 34	which handles the POST “/rest-api/signup” API call.
apps/backend/test/unit/auth/auth.controller.unit.ts - at line 71	Test the login() function in auth.controller.ts which handle the “/rest-api/login” API call
apps/backend/test/unit/courses/course.service.unit.ts - at line 28	Test the getCourses() in course.service.ts function which should return an array of CourseDTO objects.
apps/backend/test/unit/degrees/degree.service.unit.ts - at line 28	Test the getDegrees() function in degree.service.ts which should return an array of DegreeDTO objects.
apps/backend/test/unit/packages/domain/roadmap.model.unit.ts - at line 37	Test the isValid() function in degree.service.ts which checks if a roadmap is a valid roadmap.
apps/backend/test/unit/profile/profile.controller.unit.ts - at line 55	Test the getUserProfile() function in profile.controller.ts which handles the GET “/rest-api/profile” API call.
apps/backend/test/unit/profile/profile.controller.unit.ts - at line 86	Test the update() function in profile.controller.ts which handles the POST “/rest-api/profile” API call.
apps/backend/test/unit/user/user.controller.unit.ts - at line 49	Test the findOne() function in user.course.controller.ts which handles the GET “/rest-api/user/search” API call.
apps/backend/test/unit/user/user.controller.unit.ts - at line 109	Test the findAll() function in user.course.controller.ts which returns an array of users.
apps/backend/test/unit/term/term.controller.unit.ts - at line 106	Test the findCurrent() function in term.controller.ts which handles the GET “/rest-api/term/ searchCurrent” API call.

List the 5 most important integration. on tests with links below (if there are more than one unit tests in one test file, indicate clearly).

Test File path with clickable GitHub link	What is it testing (1 line description)
apps/backend/test/e2e/roadmap/personal.roadmap.service.e2e.ts - at line 22	Test the function getPersonalRoadmap() in personal.roadmap.service.ts integrated with the database which should return the Roadmap object for the current user.
apps/backend/test/e2e/section/section.controller.e2e.ts	Test the findAll() function in term.service.ts integrated with the database which should return an array of TermDTO objects.
apps/backend/test/e2e/term/term.service.e2e.ts	Test for all the courses in a term in the Term service
apps/backend/test/e2e/userCourse/userCourse.service.e2e.ts Line 59-64	Test the active courses for an invalid input for the userCourse service
apps/backend/test/e2e/userCourse/userCourse.controller.e2e.ts Line 43-48	Checks for the current courses for logged in user in the userCourse controller

List the 5 most important acceptance tests with links below. If your acceptance tests are done manually, you should have detailed steps on how to run the test and the expected outcome for test.

Test File path (if you automated the test) or as comments in Github issues (if it is with clickable GitHub link)	Which user story is it testing
Acceptance Tests for Account Management feature · maingockien01/comp4350-team8 Wiki (github.com)	US#1,2,9
Acceptance Tests for Calendar feature · maingockien01/comp4350-team8 Wiki (github.com)	US#3,6
Acceptance Tests for Course Management feature · maingockien01/comp4350-team8 Wiki (github.com)	US#5,7,8
Acceptance Tests for Roadmap feature · maingockien01/comp4350-team8 Wiki (github.com)	US#4,10

Regression testing

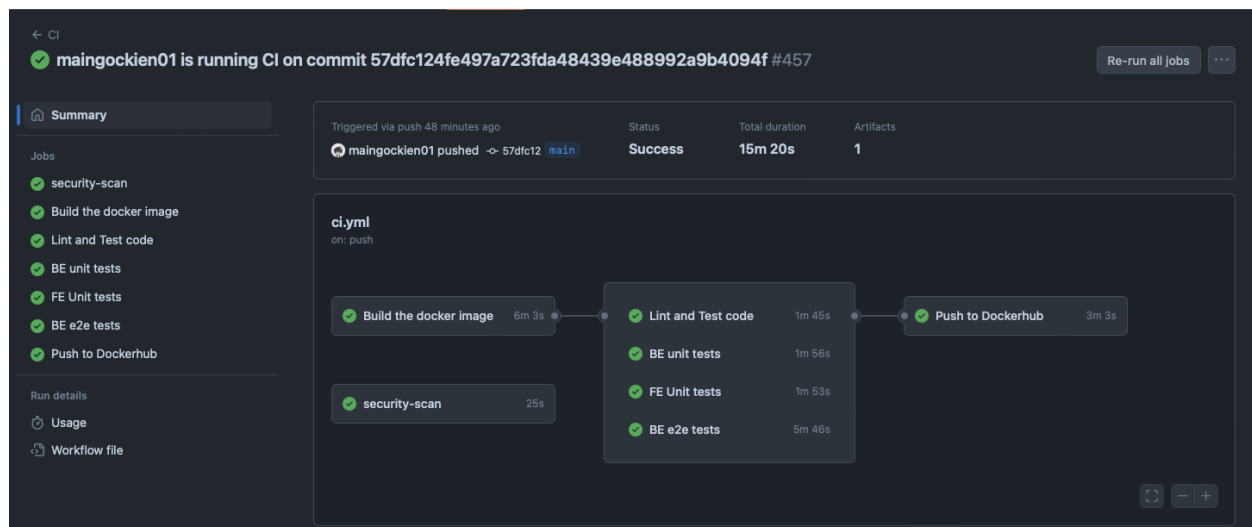
- 1) Describe how you run the regression testing (e.g., which tests are executed for regression testing and which tool is used?). 2) Provide the link to regression testing script and provide last snapshot of the execution and results of regression testing.

The regression testing run in the CI pipeline for every PR and every merge/push on main branch. We run backend unit tests, e2e tests, frontend unit tests in every CI run.

The regression run in GitHub actions. We use Jest to run unit and e2e tests in the backend. We use React test library to run frontend tests.

Link to testing script (we run test jobs in CI, hence we provide link to CI workflow here):

<https://github.com/maingockien01/comp4350-team8/blob/main/.github/workflows/ci.yml>



As you see, there are 3 test jobs in the CI.

Load testing

1) Describe the environment for load testing, such as tools and load test cases.

Tool: Locust (Python). We use 2 containers: one runs Locust, and one runs our App. The container which runs Locust will connect to the App to do the Load testing.

Load Test Cases: All API endpoints that Users will interact with. This includes Registering, Logging In, Loading Main/Look Up/List of Courses/Course Detail/Add and Drop/ Calendar/Roadmap and Personal Roadmap Page, add courses/roadmap, drop courses/roadmap, modify Account profile.

-> Average response time should be less than 300ms for all these test cases, with an average of 100 users.

2) Provide the test report for load testing.

Test Report link to the HTML file: https://github.com/maingockien01/comp4350-team8/blob/main/docs/load_test_report/report_1712711710.0159616.html

3) discuss one bottleneck found in the load testing.

There are 2 bottlenecks found in our load testing for our project released on Azure.

The 1st bottleneck is when all users register and login at the same time (at the beginning of the test), this will cause the server to become super laggy. Other than that, after some time, the server will response under 300ms time on average for any type of requests.

The 2nd bottleneck is to make sure average response time of our server to be less than 300ms, our server can only handle <= 120 users concurrently. Any number of users larger than this will result in average

response time > 300ms.

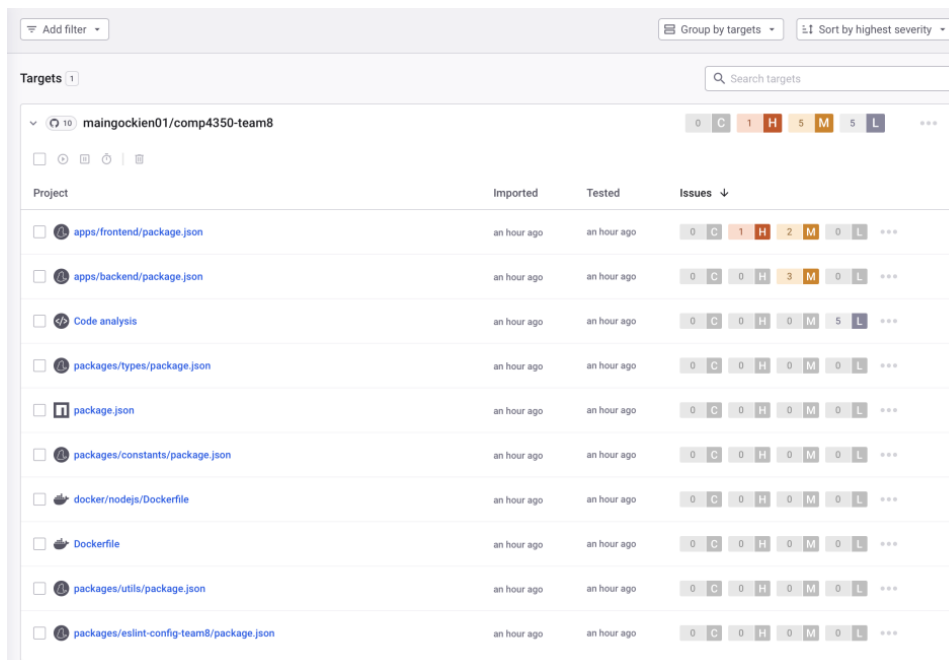
Security analysis

1) Describe the choice of the security analysis tool and how do you run it. Your security analysis tool should analyze the language used in most of your source code.

We use Snyk as our security analysis tool. It is implemented in the CI and will run for each push/pull request. The result of scanner will be exported into the Security Tab on Github.

2) Attach a report as an appendix below from static analysis tools by running the security analysis tool on your source code. Randomly select 5 detected problems and discuss what you see. Note that you are not required to fix the alarms (bugs and vulnerabilities) in the course.

Report: Since our code was all through with ESLint, there were minimal problems with it with only 5 low risk detected problems in code review scanning. All 5 of it was all related to Hardcoded Credentials as shown in the images below. The reason why this happens is because we were hardcoding some credentials so it is more easy to test our server, this happens specifically in Unit Test cases. So in our opinion, since those errors all happen only in the Unit Tests case for testing purposes, it won't impact our project and code styling/security in general.



The screenshot displays the Snyk Security Tab interface. At the top, there are controls for 'Add filter', 'Group by targets', and 'Sort by highest severity'. Below this, a search bar is labeled 'Search targets'. The main section is titled 'Targets 1' and shows a list of targets under the heading 'maingockien01/comp4350-team8'. Each target row includes a checkbox, a project name, 'Imported' and 'Tested' timestamps (all 'an hour ago'), and a summary of issues categorized by severity: Critical (C), High (H), Medium (M), and Low (L). The issues are as follows:

Target	Imported	Tested	C	H	M	L
apps/frontend/package.json	an hour ago	an hour ago	0	1	2	0
apps/backend/package.json	an hour ago	an hour ago	0	0	3	0
Code analysis	an hour ago	an hour ago	0	0	0	5
packages/types/package.json	an hour ago	an hour ago	0	0	0	0
package.json	an hour ago	an hour ago	0	0	0	0
packages/constants/package.json	an hour ago	an hour ago	0	0	0	0
docker/nodejs/Dockerfile	an hour ago	an hour ago	0	0	0	0
Dockerfile	an hour ago	an hour ago	0	0	0	0
packages/utils/package.json	an hour ago	an hour ago	0	0	0	0
packages/eslint-config-team8/package.json	an hour ago	an hour ago	0	0	0	0

☐	☒	Use of Hardcoded Credentials	Note	Test	main
#20 closed as used in tests 30 minutes ago • Detected by SnykCode in apps/.../profile/profile.controller.unit....115 43 minutes ago					
☐	☒	Use of Hardcoded Credentials	Note	Test	main
#19 closed as used in tests 30 minutes ago • Detected by SnykCode in apps/.../profile/profile.controller.unit....93 43 minutes ago					
☐	☒	Use of Hardcoded Credentials	Note	Test	main
#18 closed as used in tests 30 minutes ago • Detected by SnykCode in apps/.../user/user.controller.unit.ts:132 43 minutes ago					
☐	☒	Use of Hardcoded Credentials	Note	Test	main
#17 closed as used in tests 30 minutes ago • Detected by SnykCode in apps/.../user/user.controller.unit.ts:116 43 minutes ago					
☐	☒	Use of Hardcoded Credentials	Note	Test	main
#16 closed as used in tests 30 minutes ago • Detected by SnykCode in apps/.../user/user.controller.unit.ts:55 43 minutes ago					

 **Dismissed**

33 minutes ago

apps/backend/test/unit/user/user.controller.unit.ts:132

[Test](#)

```
129     uid: 1,
130     fullName: 'Jack Nguyen',
131     username: 'jacknguyen',
132     hashPassword: 'jack123',
```

Do not hardcode passwords in code. Found hardcoded password used in {0}.

SnykCode [Show paths](#)

```
133     pictureProfile: 'default',
134   },
135   };
```

Tool	Rule ID
SnykCode	javascript/NoHardcodedPasswords/test

Developers may use hardcoded credentials for convenience when coding in order to simplify their workflow. While they are responsible for removing these before production, occasionally this task may fall through the cracks. This also becomes a maintenance challenge when credentials are re-used across multiple applications.

[Show more](#)